

Challenge 9 - This is Yancode (Easy)

mercredi 20 mai 2026 00:49

```
hacker@reverse-engineering-this-is-yancode-easy:/challenge$ ./babyrev-level-12-0
[+] Welcome to ./babyrev-level-12-0!
[+] This challenge is an custom emulator. It emulates a completely custom
[+] architecture that we call "Yan85"! You'll have to understand the
[+] emulator to understand the architecture, and you'll have to understand
[+] the architecture to understand the code being emulated, and you will
[+] have to understand that code to get the flag. Good luck!
[+]
[+] This is an introductory Yan85 level, where we trigger Yan85 architecture
[+] operations directly. The parts of Yan85 that are used here is the emulated
[+] registers, memory, and system calls.
[+]
[+] This is a *teaching* challenge, which means that it will output
[+] a trace of the Yan85 code as it processes it. The output is here
[+] for you to understand what the challenge is doing, and you should use
[+] it as a guide to help with your reversing of the code.
[+]
[+]
```

```
var_8= qword ptr -8
; _unwind {
endbr64
push    rbp
mov     rbp, rsp
sub     rsp, 0B0h
mov     [rbp+var_A8], rdi
mov     rax, fs:28h
mov     [rbp+var_8], rax
xor     eax, eax
mov     rax, [rbp+var_A8]
add     rax, 3Ch ; '<'
mov     edx, 8 ; nbytes
mov     rsi, rax ; buf
mov     edi, 0 ; fd
call    _read
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+5Ch], 2Fh ; '/'
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+5Dh], 42h ; 'B'
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+5Eh], 75h ; 'u'
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+5Fh], 0EEh ; 'i'
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+60h], 0FCh ; 'ü'
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+61h], 59h ; 'Y'
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+62h], 0CAh ; 'Ê'
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+63h], 35h ; '5'
mov     rcx, [rax+3Ch]
mov     rax, [rbp+var_A8]
add     rax, 5Ch ; '\'
```

```
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+5Fh], 0EEh
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+60h], 0FCh
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+61h], 59h ; 'Y'
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+62h], 0CAh
mov     rax, [rbp+var_A8]
mov     byte ptr [rax+63h], 35h ; '5'
mov     rcx, [rax+3Ch]
mov     rax, [rbp+var_A8]
add     rax, 5Ch ; '\'
```

on remarque dans le programme le procédé suivant :

```
[rax+5Ch] (buffer = @92) = 2Fh ('/')
[rax+5Dh] (buffer = @93) = 42h ('B')
[rax+5Eh] (buffer = @94) = 75h ('u')
[rax+5Fh] (buffer = @95) = 0EEh ('i')
[rax+60h] (buffer = @96) = 0FCh ('ü')
[rax+61h] (buffer = @97) = 59h ('Y')
[rax+62h] (buffer = @98) = 0CAh ('Ê')
[rax+63h] (buffer = @100) = 35h ('5')
```

memcmp

[rax+3Ch] = @60 ('60') dans rcx (valeur qu'on va comparer avec la valeur se situant à l'@ 5Ch)

[rax+5Ch] = @92 ('92')

Donc le but est de mettre ses mêmes constantes car le programme va comparer notre buffer avec ses constantes : EASY

le programme compare les éléments du buffer à partir de l'@ buffer+60, et celle de buffer+92

```
python3 -c "import sys;
sys.stdout.buffer.write([0x2F,0x42,0x75,0xEE,0xFC,0x59,0xCA,0x35
])" | ./chall
```

